UNIVERSITY OF LEEDS

COMP1850 Programming

Week 2, Session 2

Booleans & Selection Statements

UNIVERSITY OF LEEDS

Today's Aims

- Understand the features of the `bool` type
- Be able to construct and evaluate **boolean expressions**
- Learn how boolean expressions are used in **selection statements** that choose between different execution paths in a program

UNIVERSITY OF LEEDS

The Boolean Type

- `bool` is a specialized version of the `int` type, supporting only two possible values
- These values have special names: `True` and `False`

True: 1
False: 0

- Note: **no quotes** (these are not strings)
- Note: **uppercase initial letter** (in other languages such as C, C++, C#, Java and Kotlin, the initial letter is lowercase)

UNIVERSITY OF LEEDS

Boolean Expressions

- A **boolean expression** evaluates to `True` or `False`
- These expressions often use **comparison operators** to compare two values in some way

Operator	Name
==	equal to
!=	not equal to
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to

Examples

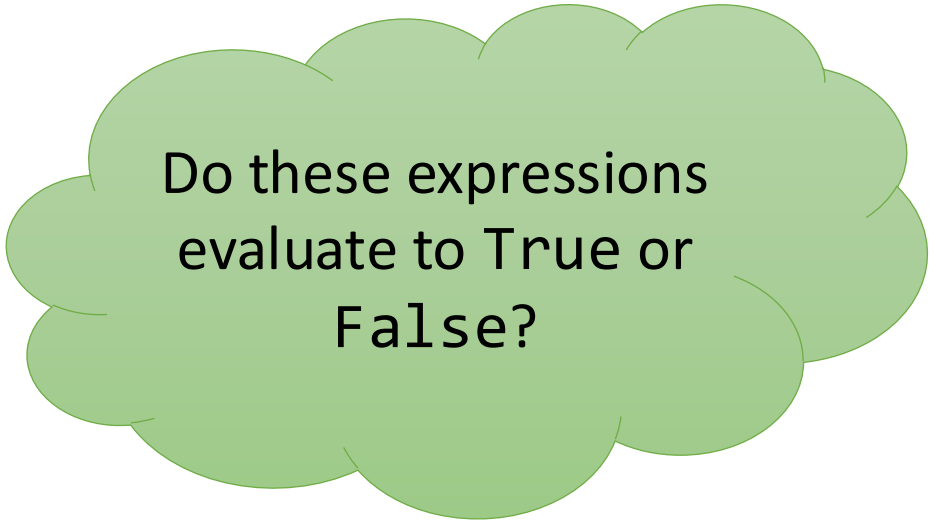
```
7 > 4

9 < 1

8 >= 3

"hello" == "Hello"

"a" > "e"
```



UNIVERSITY OF LEEDS

Comparison of Floats

```
a = 0.15 + 0.15
b = 0.1 + 0.2
a == b                  # True or False?
```

Try this out for yourself in the Python interpreter!

(See https://floating-point-gui.de/errors/comparison/ for more info)

UNIVERSITY OF LEEDS

Boolean Operators

Python has three **boolean operators** – `and`, `or`, `not` – that can be used to construct more complex boolean expressions

Expression	Result
`x and y`	`True` if both `x` and `y` are `True`, otherwise `False`
`x or y`	`True` if `x` or `y` or both are `True`, otherwise `False`
`not x`	`True` if `x` is `False`, `False` if `x` is `True`

Example

We could test whether `number` is in the range 1 to 10 with

```
number >= 1 and number <= 10
```

Note: we could avoid use of the `and` operator here by chaining the comparisons like so:

```
1 <= number <= 10
```

More Examples

a) `10 > 5`

b) `7 == 7`

c) `not (3 > 1)`

d) `(4 > 2) and (6 > 8)`

e) `(5 == 5) or (2 > 10)`

f) `not (10 > 5 or 3 != 3)`

g) `5 < 10 < 15`

h) `3 <= 3 < 10 and (10 == 10 or 5 > 8)`

Do these expressions evaluate to `True` or `False`?

Membership Testing

The `in` operator can be used to construct boolean expressions that test whether an item is part of a string or a collection:

```
"x" in "hello"                    # False
"he" in "hello"                   # True
3 in [1, 2, 3, 4]                 # True
"x" in {"a", "b", "c"}            # False
"c" in {"a": 1, "b": 2, "c": 3}   # True (tests keys)
```

UNIVERSITY OF LEEDS

Selection Statements

- A boolean expression represents a binary decision
- We can use the outcome of such an expression to choose between different paths of execution in a program
- We call statements that do this **selection statements**
- The simplest example is the **`if` statement**

UNIVERSITY OF LEEDS

Basic If Statement

`if` *boolean-expression*`:`

Block of code to execute if result is True

IMPORTANT: the block of code must consist of statements with exactly the same level of indentation!

UNIVERSITY OF LEEDS

Example

```
password = input("Enter a password: ")

if len(password) < 10:
    print("Password too short!")
    print("Minimum password length is 10")
```

UNIVERSITY OF LEEDS

If-Else Statement

```
if boolean-expression:
    Block of code to execute if result is True
else:
    Block of code to execute if result is False
```

UNIVERSITY OF LEEDS

Task 1: Simple If Statements

- Go to `week_02/session_2/tasks/`
- Edit `task_1.py` and finish the program, using the information in the comments as a guide
- Note: you will need to replace XXX with an appropriate boolean expression

UNIVERSITY OF LEEDS

Scenario

A shop gives a 10% discount when people spend £50 or more.

The till needs to decide whether people qualify for the discount and apply it automatically:

```
if cost >= 50:
    final_cost = 0.9 * cost
```

But what if the shopkeeper wants to add a *second* discount: if you spend £200 or more you get 20% off. How can we implement this?

Adding More Branches

UNIVERSITY OF LEEDS

```
if first-boolean-expression:
    Block of code to execute if first result is True
elif second-boolean-expression:
    Block of code to execute if second result is True
else:
    Block of code to execute if neither result is True
```

when the 'elif' conditions is true then after excuting the opration under the elif, the whole if-loop is terminated.

UNIVERSITY OF LEEDS

Task 2: Multiple Branches

- Go to `week_02/session_2/tasks/task_2/`
- Finish implementing each of the three programs in this directory, using the information in the comments as a guide

UNIVERSITY OF LEEDS

Common Errors

- Using `>` when `<` was needed (or vice versa)
- Choosing correctly between `>=` and `>` (or `<=` and `<`)
- Using `and` when `or` is needed (or vice versa)
- `not (A and B)` **is not the same as** `not A and not B` (see De Morgan's laws)
- Getting indentation wrong in if statements

UNIVERSITY OF LEEDS

Task 3: More Practice

- Go to `week_02/session_2/tasks/task_3/`
- Finish implementing each of the four programs in this directory, using the information in the comments as a guide

UNIVERSITY OF LEEDS

Task 4: Something Larger

`week_02/session_2/tasks/task_4/`

The file `monitor.py` is a larger program that builds on what you have seen already. Read the comments and implement Steps 1 to 4.

If you already know Python fairly well, implement the extension feature described in the comments.

Other things to try:

- Read values from a file
- Colour the output text (e.g., red for danger)